ZETTENGINE: A CXL Type-2 FPGA Architecture for GPU Memory Expansion with RISC-V Remote Processing

zett.ai

Bay Area, USA

Abstract

Modern AI systems are increasingly constrained by memory capacity and cost rather than raw compute throughput. While GPU architectures continue to scale compute density, HBM capacity remains limited by package constraints and cost scaling, resulting in underutilized compute and constrained context windows for large language models.

We implemented ZETTENGINE, a dual-node architecture that extends GPU-accessible memory using a VR2 FPGA implementing a CXL Type-2 device. The design integrates a high-throughput DMA fabric, a programmable RISC-V remote processing unit, and a memory routing subsystem that bridges GPU-visible PCIE BAR space to local and remote CXL memory. Unlike passive memory expanders, ZETTENGINE embeds a control plane inside the memory fabric, enabling scheduling, lookup, and data movement decisions to be executed in situ. The system uses a hetGPU runtime with one-to-one PTX mapping to preserve GPU execution semantics while enabling transparent redirection of memory accesses into expanded memory tiers.

1. Background and Motivation

Modern AI workloads are increasingly bottlenecked by memory capacity rather than raw compute. A high-end GPU such as the H100 ships with 80 GB of HBM, yet a single 70B-parameter model requires over 140 GB at fp16 precision. Running longer context windows or larger mixture-of-experts models compounds this further: KV-cache alone can consume tens of gigabytes at sequence lengths beyond 128K tokens. Simply buying more GPUs shifts the cost without solving the capacity problem per device.

Existing workarounds—CPU-side offloading (FlexGen [5]), unified memory, or host-managed paging—extend capacity by spilling data to DRAM or NVMe, but they stall the GPU while the CPU orchestrates each transfer. The fundamental issue is that the extended memory is treated as a *passive* resource: the CPU decides what moves and when, adding software-managed latency on top of the physical transfer cost.

CXL (Compute Express Link) provides a hardware path to attach additional DRAM directly to the PCIE fabric with cache-coherent, load/store semantics. However, commercially available CXL expanders—including those from Penguin Solutions, Enfabrica, and Liqid—remain passive: they add capacity but offer no intelligence about *which* data should be where or *when* it should arrive. This gap motivates ZETTENGINE: a memory expansion node that embeds its own processor, enabling data placement and prefetch decisions to be made inside the memory fabric itself, adjacent to the data, without burdening the host CPU or stalling the GPU.

2. System Architecture

ZETTENGINE is organized as a dual-node system: a **compute node** housing one or more GPUs, and a **memory expansion node** housing a VR2 FPGA and external DDR5/CXL-attached DRAM. The two nodes are connected via an MCIO cable carrying a PCIE Gen5 x16 link, giving a raw bandwidth ceiling of 64 GB/s in each direction.

The FPGA in the expansion node acts as a CXL Type-2 device—meaning it does not merely expose raw DRAM (Type-1), but also participates in cache-coherence with the GPU, so GPU threads can issue normal load/store instructions into the expanded address space without any software indirection. Internally, the FPGA translates these PCIE transactions into its own AXI bus and dispatches them across three layers: (1) a **protocol layer** that terminates incoming CXL/PCIE packets, (2) a **transport layer** that moves data via a high-throughput DMA engine, and (3) a **routing layer** that decides whether a given address maps to local FPGA-attached memory or to a remote CXL expander further up the fabric.

Because the GPU sees the expansion node as ordinary memory-mapped I/O (BAR2), no GPU driver modifications are needed. The same kernel that reads from HBM can read from ZETTENGINE-managed memory using identical PTX load instructions.

3. Datapath and Transport Fabric

Data movement inside ZETTENGINE is handled by a multi-channel DMA engine (`xport_mcdma`). Rather than issuing individual small transfers, the engine is *descriptor-driven*: the RISC-V RPU (or the host) writes a list of descriptors—each describing a source address, destination address, and length—and the DMA engine executes them autonomously, freeing the RPU for other work. This is the same principle used in network NIC descriptor rings, applied here to memory-to-memory transfers across the CXL fabric.

The DMA subsystem is composed of four cooperating blocks:

Descriptor Fetch Engine: Reads batches of pending transfer descriptors from host-visible memory, amortizing per-descriptor overhead through prefetching.

Execution Engine: Issues back-to-back AXI burst transactions (up to 256 B per burst) with multiple outstanding requests in flight simultaneously, saturating the available bandwidth.

Completion Engine: Tracks which transfers have finished and generates acknowledgment metadata so upstream arbiters and the RPU can issue subsequent work without polling.

CSR Interface: Exposes a set of memory-mapped control registers so the RPU can reprogram queue depth, per-channel priorities, and interrupt thresholds at runtime—enabling dynamic QoS tuning without FPGA recompilation.

Requests from the GPU host and from the RPU's DMA-initiated prefetches share the same AXI interconnect via a priority-aware arbitration block. GPU-originated traffic is given higher priority to preserve interactive latency, while background prefetch and checkpoint traffic runs at lower priority. Internal datapaths are 512 bits wide to ensure the interconnect never becomes the bottleneck before the PCIE Gen5 x16 link (64 GB/s) saturates.

4. Memory Routing and Address Mapping

From the GPU's perspective, ZETTENGINE presents a single flat address range. Internally, the `xport_bridge` block decodes each incoming address and routes it to the correct physical destination using a single address bit as a tier selector:

- **addr[45] = 0:** Route to local DDR5 attached directly to the FPGA (lowest latency tier).
- **addr[45] = 1:** Forward to a remote CXL memory module via the peer CXL port (higher capacity, slightly higher latency).

This single-bit routing keeps the hardware simple and the decode latency minimal—it is a pure combinational comparison, adding no pipeline stages. The RPU can remap which physical pages live in which tier at runtime by updating a small page-attribute table, allowing hot data to be promoted to the local DDR tier and cold data evicted to the remote tier transparently.

Credit-based flow control between the bridge and the remote CXL port prevents receiver-side buffer overflow under burst traffic, avoiding congestion collapse without requiring the GPU to throttle its issue rate.

5. RISC-V Remote Processing Unit

The distinguishing feature of ZETTENGINE relative to passive expanders is an embedded RISC-V processor—the Remote Processing Unit (RPU)—that runs *inside* the memory expansion node. Think of it as a small server co-located with the memory: it can observe every address that the GPU requests, make intelligent decisions about data placement, and issue its own DMA transfers proactively, all without any involvement from the host CPU.

The target RPU architecture aligns with the SiFive P870-D class, providing sufficient throughput to keep pace with the DMA engine:

- Fabricated on TSMC N6, compliant with the RISC-V RV23 profile
- 64-bit, 6-wide out-of-order pipeline for high single-thread throughput
- 1024-bit vector ALUs (RVV 1.0) for batch address computation and hash lookups
- Sv39/Sv48 virtual memory support and distributed IOMMU for safe multi-tenant access
- CXL-compatible coherent interface so the RPU and GPU share a consistent view of memory

The RPU connects to the DMA engine and memory bridge over a low-latency control bus and is responsible for four classes of work:

Prefetch scheduling: Given a stream of recent GPU access addresses, the RPU predicts upcoming accesses (e.g., the next KV-cache block for the next attention layer) and issues DMA transfers to bring data from the remote CXL tier into local FPGA-attached DDR before the GPU needs it.

KV-cache placement and eviction: For long-context inference, the RPU maintains a working-set map of which sequence tokens are in the hot tier and evicts the least-recently-used blocks when capacity is under pressure.

Lookup acceleration: Embedding table or sparse-feature lookups can be offloaded entirely to the RPU, which returns only the results—avoiding transferring large embedding matrices over the PCIE link.

Descriptor generation: The RPU generates DMA descriptors dynamically, enabling it to adapt transfer granularity based on observed access patterns rather than using fixed-size transfers.

6. GPU Execution Model and Compiler-Directed Acceleration

One-to-one PTX mapping. The software stack is built on the hetGPU runtime [1], which compiles GPU kernels to PTX with a one-to-one instruction mapping: every GPU load/store instruction in the original kernel maps to exactly one PTX memory operation, with no additional software-managed paging layer inserted between the GPU and the expanded memory. This means the GPU's hardware memory-access pipeline operates end-to-end, and ZETTENGINE serves each request through the CXL path without any software trap or page-fault handler.

Compiler-directed prefetch. The hetGPU compiler performs static dataflow analysis on the kernel to identify memory access sequences with predictable stride or reuse patterns. For these accesses, the compiler emits *prefetch hint annotations* targeting the expanded memory tiers. At runtime these annotations are dispatched to the RPU ahead of the consuming instruction, giving the DMA engine a head start of tens of microseconds to pull data from the remote CXL tier into local FPGA-attached DDR before the GPU warp actually issues the load. This converts what would be a synchronous stall into an asynchronous overlap of compute and data movement.

Compute-transfer overlap and bypass. The compiler schedules prefetch descriptors several GPU warp-groups ahead of their use, creating a pipeline: while the current warp-group runs matrix-multiply on data already in HBM, the DMA engine is concurrently fetching the *next* warp-group's data from the expansion node. Because the CXL link is independent of the GPU's compute units, the two proceed in parallel. For access patterns that are entirely sequential (e.g., large KV-cache reads), the compiler can additionally route traffic through a *bypass path*—a direct DMA channel from the expansion node to the GPU's HBM that circumvents the host CPU's memory controller, eliminating the additional hop through host DRAM and reducing effective latency by the CPU-NUMA penalty.

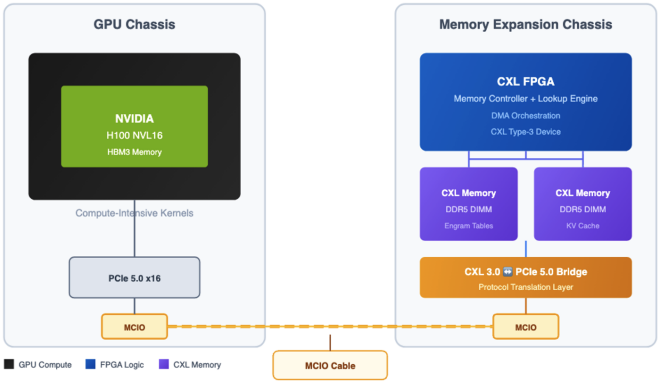

Figure 1: ZettEngine dual-node architecture. The VR2 FPGA implements the CXL Type-2 endpoint, DMA transport, RPU, and memory routing between GPU BAR space and local/remote memory.

Expert separation for MoE workloads. For mixture-of-experts (MoE) models, only a small subset of expert layers is activated per token. The compiler annotates each expert's weight tensors with a *tier preference*: active experts are pinned to local FPGA-attached DDR (or promoted to HBM), while inactive experts are evicted to the remote CXL tier. The RPU enforces these annotations through its page-attribute table, mirroring the expert-offloading strategy of KTransformers [1] but executing it entirely in hardware without host-CPU mediation.

7. Use Cases and Deployment

ZETTENGINE supports two primary deployment modes.

In standalone mode, the system integrates a RISC-V CPU complex, domain-specific matrix acceleration (PACC), and GPU compute into a unified node. The FPGA acts as both interconnect and memory controller, enabling a compact AI system with disaggregated memory.

In expansion mode, the system functions as a memory appliance attached to existing GPU servers. The CXL Type-2 interface allows it to extend GPU memory capacity without requiring modifications to the GPU hardware.

This is particularly relevant for long-context inference workloads, where KV-cache size scales linearly with sequence length.

8. Discussion

The architectural shift proposed by ZETTENGINE is to move from passive memory expansion to programmable memory systems. By embedding compute capability within the memory fabric, the system enables fine-grained control over data movement and placement.

The combination of a CXL Type-2 endpoint, high-bandwidth DMA transport, and RISC-V control plane creates a new design point between traditional GPU-local memory and host-managed offload.

Several commercial systems pursue GPU memory expansion through CXL but remain purely passive. Penguin Solutions (SMART Modular) offers CXL Type-1/2 add-in cards and E3.S modules that extend host-visible DRAM capacity [4]. Enfabrica's EMFASYS pairs CXL 2.0 DDR5 memory pooling with 800 GbE RDMA fabric [2]. Liqid's composable infrastructure couples a PCIE Gen5 switch fabric with CXL memory disaggregation [3]. None of these systems embeds a programmable control plane inside the memory fabric; placement, prefetch, and eviction decisions are delegated to the host CPU or software stack, introducing additional latency.

ZETTENGINE diverges from passive expanders by co-locating compute with memory. The hetGPU compiler hides data-movement latency by overlapping CXL transfers with GPU compute waves, yielding effective bypass of host-managed paging overhead. For mixture-of-experts workloads, the RPU can enact expert-aware data placement—keeping active expert weights in local FPGA-attached DRAM while evicting inactive experts to remote CXL tiers, analogous to the expert-offloading strategy in KTransformers [1] but executed entirely in hardware.

Beyond capacity and performance, the programmable memory node also supports fault tolerance. The RISC-V RPU can coordinate incremental checkpoint captures by streaming model state directly from GPU HBM through the DMA fabric to CXL-attached persistent memory, decoupled from the host CPU. This follows the reliability model of hetGPU+, which demonstrated that embedding checkpoint logic close to the memory subsystem substantially reduces checkpoint overhead and recovery latency compared to software-managed schemes.

9. Conclusion

This paper provides a scalable solution toward memory-centric AI systems by decoupling memory capacity and control from accelerator devices. We develop ZETTENGINE, a hardware architecture for GPU memory expansion that integrates protocol termination, transport, and programmable control within a single FPGA-based platform. This design enables direct GPU access to disaggregated memory while providing a control plane for memory orchestration. We hope this work can help establish a design paradigm for the future memory-centric AI systems.

References

[1] Azure-AI-Infrastructure. Ktransformers: A flexible framework for experiencing cutting-edge llm inference optimizations. *GitHub Repository*, 2024. `https://github.com/kvcache-ai/ktransformers`.

[2] Enfabrica. Enfabrica unveils industry's first ethernet-based AI memory fabric system for efficient superscaling of LLM inference. Press Release, 2025. Elastic Memory Fabric System (EMFASYS): CXL 2.0 DDR5 memory pooling with 800 GbE RDMA fabric.

[3] Liqid. Liqid unveils next-generation composable infrastructure solutions to scale enterprise AI for on-premises datacenter and edge environments. BusinessWire Press Release, 2025. EX-5410C CXL memory and EX-5410P GPU chassis with PCIe Gen5 and CXL switch fabric.

[4] Penguin Solutions. SMART modular technologies showcases cutting-edge CXL memory solutions at AI infra summit. Product Brief and Press Release, 2025. Passive CXL Type-1/2 add-in cards and E3.S memory modules for capacity expansion.

[5] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pages 31094–31116. PMLR, 2023.